

DA3452 – Speech and Language Technology

Weekend Reading and Coding Assignment 2

Normalization, BPE, Corpora, Bag-of-Words, and TF-IDF

Status of this assignment. This sheet is **not for submission**. However, every student is responsible for the reading, concepts, hand calculations, and coding exercises described here. The material may be tested in the Module 1 assessment and the end-semester examination.

By the end of this work, you should be able to:

- explain why Unicode normalization is needed and apply NFC normalization in Python;
- distinguish Unicode normalization from task-specific text normalization;
- train a small character-level BPE tokenizer and use its fixed merge rules to encode unseen words;
- explain the distinction between character-level and byte-level BPE;
- construct Bag-of-Words document vectors and a document–term matrix;
- compute log-weighted TF, DF, IDF, and TF-IDF using the course convention; and
- explain what these representations preserve, collapse, and discard.

Scope relative to Assignment 1. Assignment 1 already covered Jurafsky–Martin Sections 2.1–2.3 and the UTF-32/UTF-8 encoding exercises. This sheet starts with material that was **not assigned in Assignment 1** and completes the independent-study work for Module 1.

Lecture 6 note. The n-gram language-model material begun in Lecture 6 belongs to **Module 2**. It is not part of this Module 1 weekend assignment and is not part of the Module 1 test boundary.

Part A: Required reading

Read the following portions of Daniel Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd edition draft.

Chapter 2 – Words and Tokens

- **Section 2.4 – Subword Tokenization: Byte-Pair Encoding**
 - Section 2.4.1 – BPE training
 - Section 2.4.2 – BPE encoder
 - Section 2.4.3 – BPE in practice
- **Section 2.5 – Corpora**

Chapter 11 – Retrieval-based Models

Read only the following parts of Section 11.1:

- **Section 11.1.1 – Representing documents as vectors**
- **Section 11.1.2 – Term weighting**, through the definition of TF-IDF in **Eq. 11.6**

Stop after TF–IDF. Do not continue to Section 11.1.3 (Document Scoring) for this assignment or for Module Test 1. Cosine similarity has been used in class for intuition, but detailed document scoring is outside the Module 1 test boundary.

Lecture material to review alongside the reading

The following ideas were developed in class and are also examinable, even where the textbook treatment is brief or different in emphasis:

- canonical equivalence and NFC Unicode normalization;
- Unicode normalization versus task-specific normalization;
- tokenizer training versus tokenizer encoding;
- character-level versus byte-level BPE;
- vocabulary size versus sequence length;
- token fertility and why it can differ across languages;
- sparsity of Bag-of-Words representations; and
- what Bag-of-Words and TF–IDF preserve, collapse, and discard.

Questions you should be able to answer after reading

Prepare brief answers, in your own words, to the following questions:

1. Why does subword tokenization help with unseen words?
2. What does the BPE *trainer* learn, and what does the BPE *encoder* do with a new input?
3. Why must learned BPE merge rules be applied in their learned order?
4. Why are BPE subwords not necessarily linguistic morphemes?
5. What changes when BPE starts from UTF-8 bytes rather than Unicode characters?
6. What is gained and lost as the BPE vocabulary becomes larger?
7. Why can multilingual tokenizers use more tokens for the same amount of content in one language than in another?
8. Why should corpus statistics be interpreted as properties of a particular collection rather than universal properties of a language?
9. In a Bag-of-Words model, what determines the dimension of a document vector?
10. Why can a short document have a very high-dimensional but sparse vector?
11. What is the difference between term count, document frequency, and term frequency as used in this course?
12. Why does a term occurring in every document receive IDF equal to zero under the course convention?
13. What information can TF–IDF reweight, and what information can it not recover once Bag-of-Words has discarded it?

Part B: Unicode normalization in Python

Two strings may look identical while containing different code-point sequences. Work with the following pair:

```
a = "caf\u00E9" # precomposed e-acute
b = "cafe\u0301" # e followed by combining acute accent
```

Write a short program that, **before normalization**, prints:

1. the two strings;
2. whether `a == b`;
3. `len(a)` and `len(b)`; and
4. the Unicode code points in each string in U+XXXX notation.

Then normalize both strings using NFC:

```
import unicodedata

a_nfc = unicodedata.normalize("NFC", a)
b_nfc = unicodedata.normalize("NFC", b)
```

Repeat the four checks above after normalization.

A small task-normalization experiment

Consider the following strings:

```
texts = [
    "Apple",
    "apple",
    "I'M READY!",
    "I'm ready!",
    "Rs. 1,000",
]
```

Write two simple preprocessing functions:

1. one that performs **only NFC normalization**;
2. one that performs NFC normalization and then lowercases the text.

Do **not** automatically remove punctuation, contractions, or numbers. Instead, answer:

- Which distinctions did lowercasing remove?
- Could those distinctions matter for some NLP tasks?
- Why is NFC normalization conceptually different from deciding to lowercase text?

Key distinction. Unicode normalization standardizes equivalent Unicode representations. Task-specific normalization deliberately decides which distinctions the model should keep or discard.

Part C: Train and use a small BPE tokenizer

Implement the **character-level classroom version** of BPE. Do not use a tokenizer library for this part.

Use the following weighted corpus, keeping the words in this order:

```
word_counts = [
    ("new", 2),
    ("renew", 2),
    ("set", 1),
    ("reset", 1),
]
```

Represent each word initially as a tuple of characters. For example:

```
("new", 2) -> (("n", "e", "w"), 2)
```

Step 1: Count adjacent pairs

Write a function:

```
def count_pairs(symbol_sequences):
    """
    Return weighted counts of adjacent symbol pairs.
    Each sequence has an associated word frequency.
    """
    pass
```

A pair that appears once inside a word whose corpus frequency is 2 contributes **2** to the pair count. For reproducibility, use the following tie-breaking rule:

If two pairs have the same weighted frequency, choose the pair that is encountered first when scanning the words in the order listed above, from left to right within each word.

With this rule, your first three learned merges should begin:

Step	Selected pair	New token
1	n + e	ne
2	ne + w	new
3	r + e	re

Step 2: Merge a selected pair

Write a function that replaces every non-overlapping occurrence of a selected adjacent pair by the merged token:

```
def merge_pair(sequence, pair):
    """Merge one adjacent symbol pair throughout one sequence."""
    pass
```

Test it on simple cases before using it inside the trainer.

Step 3: Train for k merges

Write:

```
def train_bpe(word_counts, k):
    """
    Train character-level BPE for k merges.
    Return the ordered merge rules and the final vocabulary.
    """
    pass
```

Train for $k = 5$. Record the ordered merge rules. Check that each training iteration:

- counts pairs using corpus frequencies;
- selects one most frequent pair;
- adds exactly one new token to the vocabulary; and
- updates the corpus representation before the next count.

Step 4: Encode an unseen word using fixed rules

Now write a separate encoder:

```
def encode_bpe(word, merge_rules):
    """
    Start from characters and apply the learned merge rules
    in their learned order. Do not learn anything new here.
    """
    pass
```

Use your learned rules to encode:

```
"newest"
"renew"
"reset"
```

For `newest`, the first two rules should produce the sequence:

n e w e s t → n e w e s t → n e w e s t

Explain why the unseen word does **not** need to be added to the vocabulary during encoding.

Character-level versus byte-level starting symbols

Run:

```
text = "\u00E9"
print([f"U+{ord(c):04X}" for c in text])
print([f"{b:02X}" for b in text.encode("utf-8")])
```

Explain what the initial symbol sequence would be for:

- a character-level BPE tokenizer; and
- a byte-level BPE tokenizer.

You are **not** required to implement a complete byte-level BPE tokenizer in this assignment.

Part D: Build Bag-of-Words document vectors

For this part, use the following small corpus:

```
documents = [
    "language models learn from language data",
    "speech models learn from speech data",
    "translation models learn from parallel data",
    "retrieval models search document data",
]
```

For this controlled exercise, assume that the strings are already normalized, lowercased, punctuation-free, and whitespace-tokenized. Do not add stemming or stop-word removal.

Step 1: Construct the feature vocabulary

Write a function that:

1. collects every distinct word type in the corpus;
2. includes each type exactly once; and
3. returns the feature vocabulary in alphabetical order.

```
def build_vocabulary(documents):
    pass
```

Answer:

- What is $|V|$ for this corpus?
- Why does `language` appear only once in the vocabulary even though it occurs twice in the first document?
- Would a different fixed vocabulary ordering change the meaning of the representation if every document used that same ordering consistently?

Step 2: Construct count vectors

Write:

```
def bow_vector(document, vocabulary):
    """Return the Bag-of-Words count vector for one document."""
    pass
```

Then construct the full **document-term matrix**: rows are documents and columns are feature-vocabulary terms.

```
def document_term_matrix(documents, vocabulary):
    pass
```

Check the first document carefully by hand before trusting your program.

Step 3: Inspect sparsity

Compute:

- the total number of matrix entries;
- the number of zero entries;
- the number of non-zero entries; and
- the fraction of entries that are zero.

Then produce a sparse dictionary representation of the first document containing only non-zero feature/-count pairs.

Answer: does sparse storage change the mathematical document vector, or only how it is stored?

Part E: Implement TF-IDF from scratch

Use the exact convention used in class and in Jurafsky–Martin Section 11.1.2.

For a term t and document d , let $c(t, d)$ be the raw count. Define log-weighted term frequency as

$$\text{tf}(t, d) = \begin{cases} 1 + \log_{10} c(t, d), & c(t, d) > 0, \\ 0, & c(t, d) = 0. \end{cases}$$

Let df_t be the number of documents containing t , and let N be the number of documents. Define

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right),$$

and

$$\text{tfidf}(t, d) = \text{tf}(t, d) \text{idf}_t.$$

Step 1: Implement the components

Write functions of the following form:

```
import math

def log_tf(count):
    pass

def document_frequency(term, tokenized_documents):
    pass

def idf(df, n_documents):
    pass
```

Then construct the TF–IDF vector for every document, using the **same feature vocabulary and ordering** as in Part D.

Step 2: Hand-check selected values

Before relying on the full matrix, calculate by hand:

1. df and idf for models;
2. df and idf for learn;
3. df and idf for language;
4. TF for language in the first document; and
5. TF–IDF for language in the first document.

Your program should agree with your hand calculations up to ordinary floating-point rounding.

Step 3: Change the corpus and reason about the result

Add a fifth document:

```
"language technology uses language models"
```

Recompute the feature vocabulary, DF, IDF, and TF–IDF values.

Answer:

1. Which terms changed document frequency?
2. Did the IDF of language increase or decrease? Why?
3. Can an existing document's TF–IDF vector change even when the document itself has not changed?
4. What does this tell you about TF–IDF being *corpus-relative*?

Do not use sklearn to implement the main calculation. The purpose is to make the representation and weighting visible. After your own implementation works, you may optionally compare it with a library implementation, taking care to check whether the library uses exactly the same TF and IDF conventions.

Part F: Representation choices and failure cases

For each pair below, state whether a unigram Bag-of-Words count vector would distinguish the two texts, and briefly explain why.

1. dog bites man versus man bites dog
2. car versus automobile
3. river bank versus bank loan
4. not good but funny versus good but not funny

Then answer:

- Which failures are caused by discarded word order or composition?
- Which are caused by using separate vocabulary coordinates for related words?
- Which are caused by collapsing different senses of the same word into one coordinate?
- Can changing TF–IDF weights alone repair these representation limitations?

Self-check before the Module 1 test

You should be able to complete all of the following without consulting this sheet:

- explain canonical equivalence and show what NFC normalization changes computationally;
- distinguish Unicode normalization from task-specific normalization;
- train a small BPE tokenizer by hand for several merge steps;
- apply a fixed ordered list of BPE merges to an unseen word;
- distinguish tokenizer training from tokenizer encoding;
- explain character-level versus byte-level BPE;
- reason about vocabulary size, sequence length, and token fertility;
- define document, corpus, and feature vocabulary;
- construct and interpret a Bag-of-Words document vector and document–term matrix;
- explain sparsity;
- compute TF, DF, IDF, and TF–IDF for a small corpus using the course convention;
- explain why a term appearing in every document receives IDF zero; and
- explain what Bag-of-Words and TF–IDF preserve, collapse, and discard.

Keep your code, hand calculations, and answers for revision. The work is not collected, but the underlying concepts and procedures are examinable.